06

Scikit-learn

The main toolbox for classical Machine Learning

Teach the computer to find patterns and make predictions

AI/ML Engineer Learning Series · Deep Dive Edition

What it is	A free Python library full of ready-made ML models
Short name	sklearn (that's what you type in code)
Best for	Classical ML — not deep learning
The magic	Every model works the same way: fit then predict
Built on	NumPy and SciPy (so it's fast)

What's Inside

Here is everything covered in this guide, in order:

1. What is Scikit-learn?

Scikit-learn (■■■■■■-■■■■■), which you write as **sklearn** in code, is a free toolbox full of ready-made machine learning models. Instead of building a model from zero, you just pick one off the shelf and use it.

Think of it like a kitchen full of appliances. You don't build a blender to make juice — you grab the blender and press the button. Sklearn gives you the 'appliances' (models), and they all have the same simple buttons: **fit** and **predict**.

It is made for **classical machine learning** (■■■■■■■■■■■ ■■■■) — things like predicting house prices, sorting emails into spam/not-spam, or grouping customers. For deep learning (big neural networks), you'd use PyTorch or TensorFlow instead — but those come later. Sklearn is where everyone starts.

> ■ *The best part: once you learn how ONE sklearn model works, you know how ALL of them work. They all share the same fit/predict pattern. Learn it once, use it everywhere.*

2. Two Main Types of ML

Before models, know the two big families of problems sklearn solves:

Supervised Learning — Learning from Answers

Supervised (■■■■■■■■■■ — ■■■■■■■■■■■■) learning is when you have the answers already. You show the model questions AND their correct answers, and it learns the pattern. There are two kinds:

- **Classification** (■■■■■■■■■■■■■) — predict a category. Example: is this email spam or not? Is this a cat or a dog?
- **Regression** (■■■■■■■■) — predict a number. Example: what price will this house sell for? How many sales next month?

Unsupervised Learning — Finding Hidden Groups

Unsupervised (■■■■■■■■■■■■) learning is when you have NO answers. You just have data, and you ask the model to find patterns or groups on its own. Example: group your customers into similar types, without telling it the types first. This is called **clustering** (■■■■■■■■■■■ — ■■ ■■■■■■■).

Type	You have answers?	Goal	Example
Classification	Yes	Predict a category	Spam or not
Regression	Yes	Predict a number	House price
Clustering	No	Find groups	Customer types

3. The Machine Learning Workflow

Almost every ML project follows the same six steps. Once you know these, every project feels familiar.

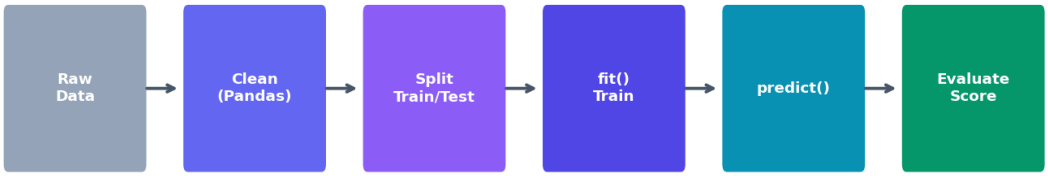

Fig 1 — The 6-step ML workflow. Clean → split → train → predict → check.

Step	What happens
1. Raw Data	Get your data (CSV, database, etc.)
2. Clean	Fix missing values and types with Pandas
3. Split	Cut data into a training part and a testing part
4. Train (fit)	Show the training data to the model so it learns
5. Predict	Ask the model to guess on data it hasn't seen
6. Evaluate	Compare guesses to real answers — how good is it?

4. The Train/Test Split

This is one of the most important ideas in ML. You split your data into two parts. The model learns from the **training set**. Then you test it on the **test set** — data it has never seen. This tells you if it really learned, or just memorised.

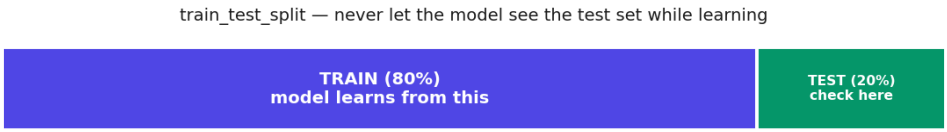

Fig 2 — Usually 80% for training, 20% for testing. The test set stays hidden during training.

Why hide some data? Imagine a student who memorises the exact answers to practice questions. They'll ace the practice but fail the real exam with new questions. The test set is the 'real exam' — it checks if the model can handle new data.

```
from sklearn.model_selection import train_test_split

# X = input features, y = the answer we want to predict
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,      # 20% goes to testing
    random_state=42)    # makes the split repeatable

print(X_train.shape, X_test.shape)
```

■ *random_state=42 just makes sure you get the SAME split every time you run the code. The number itself doesn't matter — 42 is just a popular choice.*

5. fit and predict — The Two Magic Words

Every single sklearn model works with the same two steps. This is the heart of sklearn. Learn this pattern and you can use any model.

- **.fit(X_train, y_train)** — this is training. The model looks at the data and learns the pattern. (fit ■■■■ ■■■■)
- **.predict(X_test)** — this is using it. The model makes guesses on new data. (predict ■■■■ ■■■■■■ ■■■)

```
from sklearn.linear_model import LogisticRegression

# 1. Create the model (pick one off the shelf)
model = LogisticRegression()

# 2. Train it on the training data
model.fit(X_train, y_train)

# 3. Make predictions on new data
predictions = model.predict(X_test)

# 4. Check how good it is
from sklearn.metrics import accuracy_score
print(accuracy_score(y_test, predictions))
```

The amazing thing: to try a different model, you change just ONE line:

```
from sklearn.tree import DecisionTreeClassifier

model = DecisionTreeClassifier()   # <-- only this line changed
model.fit(X_train, y_train)        # everything else is identical
predictions = model.predict(X_test)
```

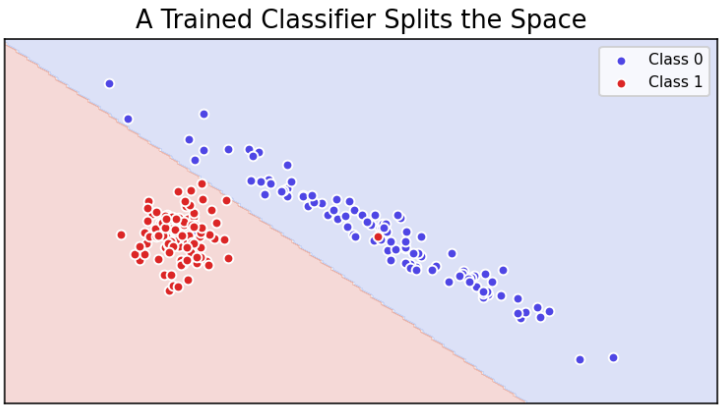

Fig 3 — After fit(), a classifier divides the space. New points are labelled by which side they fall on.

6. Your First Full Model

Let's put it all together. Here is a complete, working example predicting whether a flower is one type or another, using sklearn's built-in iris dataset.

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# 1. Load data (X = measurements, y = flower type)
X, y = load_iris(return_X_y=True)

# 2. Split into train and test
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# 3. Create and train the model
model = RandomForestClassifier()
model.fit(X_train, y_train)

# 4. Predict and check
preds = model.predict(X_test)
print('Accuracy:', accuracy_score(y_test, preds))
# Accuracy: 1.0   (got every test flower right!)
```

■ *That's a real, working ML model in about 12 lines. This same skeleton works for almost any classical ML problem — just swap the data and the model.*

7. Preprocessing — Getting Data Ready

Models only understand numbers, and they work best when numbers are on a similar scale. **Preprocessing** (■■■■■■■■■■■■ — ■■■ ■■■■ ■■■■■■■■ ■■■) means preparing the data first.

Scaling — Making Numbers Comparable

If one column is age (0-100) and another is salary (0-100000), the big numbers can bully the small ones. Scaling shrinks everything to a similar range so each column is fair.

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)  # learn + apply
X_test_scaled  = scaler.transform(X_test)       # apply only
```

■ *Notice: fit_transform on TRAIN, but only transform on TEST. The scaler must learn from training data only — never peek at the test set.*

Encoding — Turning Text into Numbers

```
from sklearn.preprocessing import LabelEncoder, OneHotEncoder

# Label encoding: 'cat','dog' -> 0, 1
le = LabelEncoder()
y_encoded = le.fit_transform(y_text)

# One-hot is usually done in Pandas:
import pandas as pd
X = pd.get_dummies(df, columns=['city'])
```

8. Measuring Success

How do you know if your model is good? You measure it. Different problems use different scores. (We go deeper on these in a later topic — here's the quick version.)

For Classification (predicting categories)

Metric	Plain meaning
Accuracy	Out of all guesses, how many were right
Precision	When it says 'yes', how often is it correct
Recall	Out of all real 'yes' cases, how many it caught
F1-score	A balance of precision and recall

```
from sklearn.metrics import (accuracy_score, precision_score,
                             recall_score, f1_score,
                             classification_report)

print(accuracy_score(y_test, preds))
print(classification_report(y_test, preds))  # all at once
```

For Regression (predicting numbers)

Metric	Plain meaning
MAE	Average size of the error (easy to understand)
MSE	Average of squared errors (punishes big misses)
RMSE	Square root of MSE (back to original units)
R^2 score	How much of the pattern the model explains (1 = perfect)

```
from sklearn.metrics import mean_squared_error, r2_score

print(mean_squared_error(y_test, preds))
print(r2_score(y_test, preds))
```

9. Overfitting — The Big Danger

Overfitting (■■■■■■■■■■) is when a model memorises the training data instead of learning the real pattern. It scores great on training data but fails on new data. The opposite, **underfitting**, is when the model is too simple to learn anything useful.

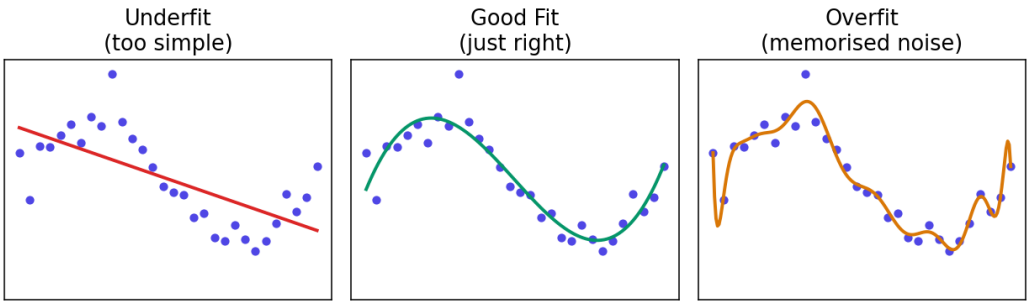

Fig 4 — Left: too simple (underfit). Middle: just right. Right: memorised every wiggle (overfit).

Problem	Training score	Test score	Meaning
Underfit	Low	Low	Too simple — learned too little
Good fit	High	High	Learned the real pattern
Overfit	Very high	Low	Memorised — can't handle new data

Ways to fight overfitting:

- Get more training data
- Use a simpler model
- Use regularization (a setting that keeps the model humble)
- Use cross-validation (next section) to catch it early

10. Cross-Validation — A Fairer Test

One train/test split can be lucky or unlucky. **Cross-validation** (■■■■-■■■■■■■■■■) tests the model several times on different splits, then averages the scores. This gives a much more trustworthy result.

The common version is **K-Fold**: cut the data into K parts (say 5). Train on 4 parts, test on 1. Repeat 5 times so each part gets to be the test once. Average all 5 scores.

```
from sklearn.model_selection import cross_val_score

model = RandomForestClassifier()
scores = cross_val_score(model, X, y, cv=5)  # 5 folds

print(scores)        # 5 scores, one per fold
print(scores.mean()) # the average — trust this one
```

11. Pipelines — Bundle Every Step

A **Pipeline** (■■■■■■■■) chains your steps together — scaling, then the model — into one object. This keeps your code clean and prevents mistakes like forgetting to scale the test data the same way.

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

pipe = Pipeline([
    ('scaler', StandardScaler()),       # step 1: scale
    ('model',  LogisticRegression())    # step 2: train
])

# Now fit and predict on the WHOLE pipeline at once
pipe.fit(X_train, y_train)
preds = pipe.predict(X_test)
# scaling is applied automatically and correctly every time
```

■ *Pipelines are a sign of clean, professional ML code. They make sure every step happens in the right order, on both training and test data.*

12. Common Mistakes

Mistake	Why it's a problem	Fix
Testing on training data	Looks great, lies to you	Always keep a separate test set
Scaling before splitting	Test info leaks into training	Split first, then scale
fit on test data	Cheating — test must stay unseen	Only transform the test set
Only using accuracy	Misleading with unbalanced data	Also check precision/recall
Ignoring overfitting	Fails on real new data	Use cross-validation
Not setting random_state	Results change every run	Set random_state for repeatability

Quick Reference Cheatsheet

Task	Code
Split data	train_test_split(X, y, test_size=0.2)
Train a model	model.fit(X_train, y_train)
Make predictions	model.predict(X_test)
Prediction probabilities	model.predict_proba(X_test)
Accuracy	accuracy_score(y_test, preds)
Full report	classification_report(y_test, preds)
Regression error	mean_squared_error(y_test, preds)
R2 score	r2_score(y_test, preds)
Scale features	StandardScaler().fit_transform(X)
Encode labels	LabelEncoder().fit_transform(y)
Cross-validation	cross_val_score(model, X, y, cv=5)
Build a pipeline	Pipeline([('scaler',..),('model',..)])
Classifier model	RandomForestClassifier()
Regression model	LinearRegression()
Decision tree	DecisionTreeClassifier()
Logistic regression	LogisticRegression()
Save a model	joblib.dump(model, 'model.pkl')
Load a model	joblib.load('model.pkl')

Next Topic: Linear Regression — the simplest and most important ML model. We'll open it up and see exactly how a model 'learns' to draw the best line through data.